



Building Java Programs
A BACK TO BASICS APPROACH 5e

Fundamentals of Computer Science

II

06 - Stack & Queue (Basic)

Instructor: Varik Hoang



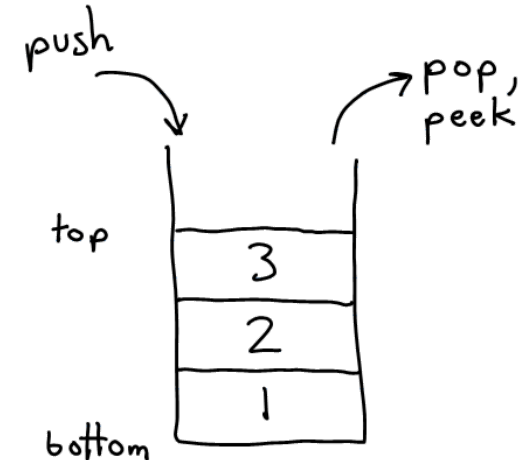
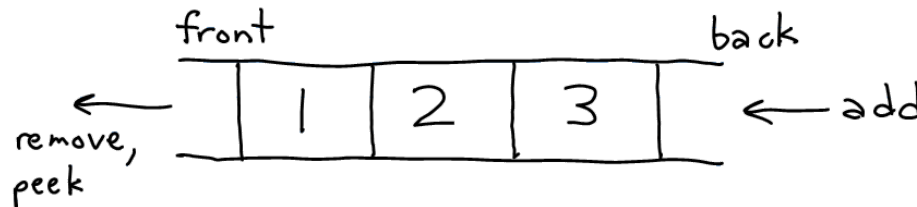
Objectives

- Stack/Queue Basics
 - Concepts
 - Operations
 - Example
 - Case Study



Stack/Queue Basics

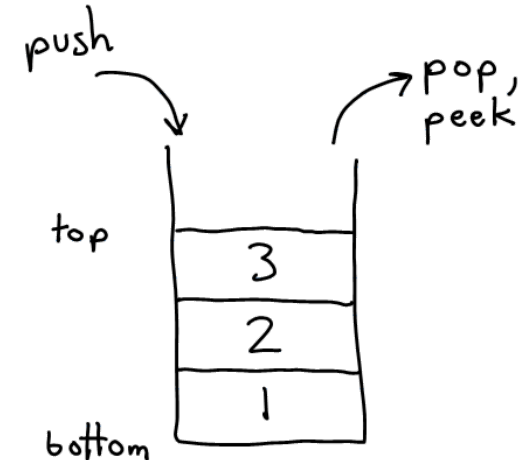
- A stack or queue stores an ordered sequence of values:
 - some way to put values into the structure ("**add**" operation)
 - a way to take values out ("**remove**" operation)
 - a way to test whether there is anything left in the structure ("**isEmpty**" check)
- A **size** method that lets you ask for the number of elements
- A **peek** method to examine the next value (without removing the value)
- **Stacks** and **Queues** are similar:
 - **Stack** has **LIFO** structures (Last-In, First-Out).
 - **Queue** has **FIFO** structures (First-In, First-Out).





Stack - Concepts

- All of the action occurs on the **top** of the stack.
- The adding operation is called a **push** and the removing operation is called a **pop**.
- We **push** values onto the **top** and we **pop** them off the top.
- There are three basic operations for **Stack**:
 - **push**: add an element on the top of the stack.
 - **pop**: remove the top element off the stack.
 - **peek**: examine the top element (does not remove).





Stack - Operations

Method	Description
Stack<Type>()	Construct a new stack with elements of type Type
push(value)	Place given value on top of stack
pop()	Remove top value from stack and return it or throws EmptyStackException if stack is empty
peek()	Return top value from stack and return it or throws EmptyStackException if stack is empty
size()	Return number of elements in stack
isEmpty()	Return true if stack has no element
clear()	Remove all values from the stack



Stack - Example

```
import java.util.*;
public class StackMain
{
    public static void main(String[] args)
    {
        Stack myStack = new Stack();
        System.out.println(myStack.isEmpty());
        System.out.println(myStack.size());
        myStack.push(10);
        myStack.push(20);
        myStack.push(30);
        myStack.push(40);
        System.out.println(myStack.isEmpty());
        System.out.println(myStack.size());
        System.out.println(myStack);
        System.out.println(myStack.peek());
        System.out.println(myStack);
        System.out.println(myStack.pop());
        System.out.println(myStack);
        myStack.clear();
        System.out.println(myStack.isEmpty());
        System.out.println(myStack.peek());
    }
}
```

```
varikmp@gazelle:~/ $ javac StackEx.java
varikmp@gazelle:~/ $ java StackEx
true
0
false
4
[10, 20, 30, 40]
40
[10, 20, 30, 40]
40
[10, 20, 30]
true
Exception in thread "main" java.util.EmptyStackException
    at java.base/java.util.Stack.peek(Stack.java:102)
    at StackMain.main(StackMain.java:22)
varikmp@gazelle:~/ $ _
```



Stack - Case Study - Find Max

— The algorithm is correct but what is wrong with the code?

```
// Precondition: myStack.size() > 0
public static int max(Stack<Integer> myStack)
{
    int maxValue = myStack.pop();

    while (!myStack.isEmpty())
    {
        int nextValue = myStack.pop();
        if (maxValue < nextValue)
            maxValue = nextValue;
    }

    return maxValue;
}
```

- We must save and restore the stack data
- The code snippet on the right is a solution

```
// Precondition: myStack.size() > 0
public static int max(Stack<Integer> myStack)
{
    Stack<Integer> backupStack = new Stack();
    int maxValue = myStack.pop();
    backupStack.push(maxValue);

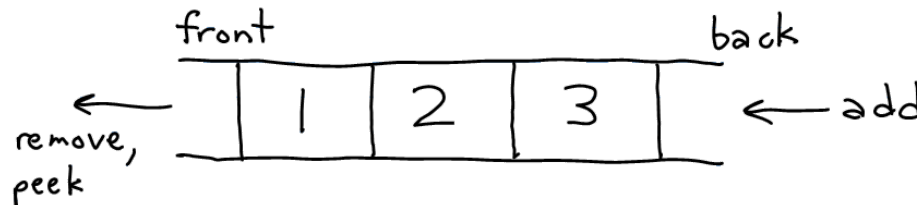
    while (!myStack.isEmpty())
    {
        int nextValue = myStack.pop();
        backupStack.push(nextValue);
        if (maxValue < nextValue)
            maxValue = nextValue;
    }

    while (!backupStack.isEmpty())
    {
        myStack.push(backupStack.pop());
    }

    return maxValue;
}
```

Queue - Concepts

- Manipulate both ends of the structure
- Add new values at the back of the queue and remove values from the front.
- The adding and removing operations are simply called add (enqueue) and remove (dequeue).
- There are three basic operations for **Queue**:
 - **add** (enqueue): add an element to the back of the queue.
 - **remove** (dequeue): remove the front element off the queue.
 - **peek**: examine the front element (does not remove).





Queue - Operations

Method	Description
Queue<Type>()	Construct a new queue (using LinkedList) with elements of type Type
add(value)	Insert given value to the back of queue
remove()	Return front value from queue and return it or throws NoSuchElementException if queue is empty
peek()	Remove front value from queue and return it or return null if queue is empty
size()	Return number of elements in queue
isEmpty()	Return true if queue has no element
clear()	Remove all values from the queue



Queue - Example

```
import java.util.*;
public class QueueMain
{
    public static void main(String[] args)
    {
        Queue myQueue = new LinkedList();
        System.out.println(myQueue.isEmpty());
        System.out.println(myQueue.size());
        myQueue.add(10);
        myQueue.add(20);
        myQueue.add(30);
        myQueue.add(40);
        System.out.println(myQueue.isEmpty());
        System.out.println(myQueue.size());
        System.out.println(myQueue);
        System.out.println(myQueue.peek());
        System.out.println(myQueue);
        System.out.println(myQueue.remove());
        System.out.println(myQueue);
        myQueue.clear();
        System.out.println(myQueue.isEmpty());
        System.out.println(myQueue.peek());
        System.out.println(myQueue.remove());
    }
}
```

```
varikmp@gazelle:~/ $ javac QueueEx.java
varikmp@gazelle:~/ $ java QueueEx
true
0
false
4
[10, 20, 30, 40]
10
[10, 20, 30, 40]
10
[20, 30, 40]
true
null
Exception in thread "main" java.util.NoSuchElementException
    at java.util.LinkedList.removeFirst(LinkedList.java:274)
    at java.util.LinkedList.remove(LinkedList.java:689)
    at QueueMain.main(QueueMain.java:22)
varikmp@gazelle:~/ $ _
```



Queue - Case Study - Sum Values

- The algorithm is correct but what is wrong with the code?

```
public static int sum(Queue<Integer> myQueue)
{
    int sumValue = 0;

    while (!myQueue.isEmpty())
    {
        int nextValue = myQueue.remove();
        sumValue = sumValue + nextValue;
    }

    return sumValue;
}
```

```
public static int sum(Queue<Integer> myQueue)
{
    int sumValue = 0;
    int size = myQueue.size();

    while (size > 0)
    {
        int nextValue = myQueue.remove();
        sumValue = sumValue + nextValue;

        myQueue.add(nextValue);
        size--;
    }

    return sumValue;
}
```

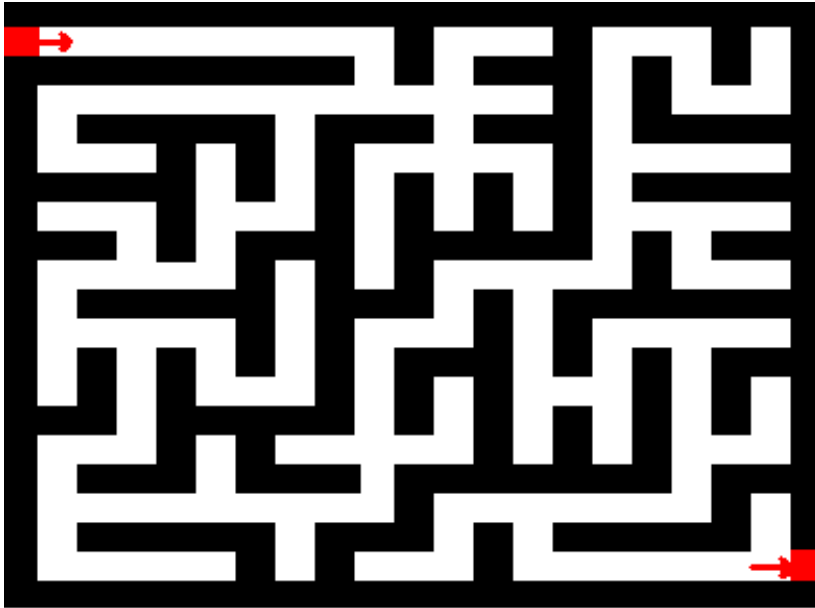
- We must capture the size
- Add up the value to the sum
- Add the value back to the queue
- The code snippet on the right is a solution



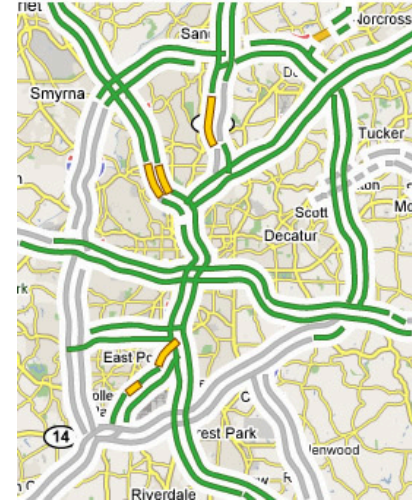
Graph Traversal

- **Depth-First-Search (DFS):**
 - An algorithm for searching a graph or tree data structure
 - Start from a vertex and goes as far as it finds an unexplored path
 - Use **Stack** and **List** to implement the algorithm
- **Breadth-First-Search (BFS):**
 - An algorithm for finding a shortest path in a graph
 - Start from a vertex and explore all neighbors that has not been visited
 - Use **Queue** and **List** to implement the algorithm
- Both algorithms do this until the entire graph has been explored

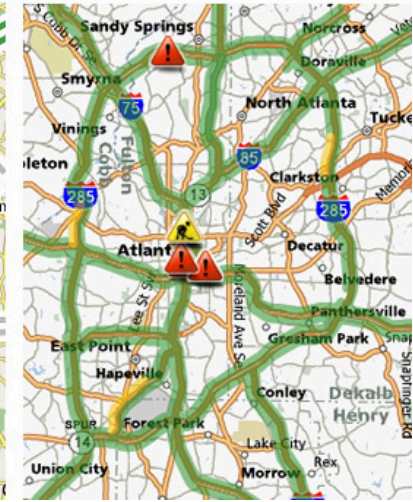
Graph Traversal - Real World Application



Google Maps



MapQuest



Questions & Answer





Thank You!

A black fountain pen with silver-colored accents, including a clip and a band near the tip. The pen is positioned diagonally, pointing towards the bottom right, and is placed just below the end of the 'Thank You!' text.